ARCHITECTING AGENTIC AI SOLUTIONS

PHYSICAL ARCHITECTURE

© 2011-25 National University of Singapore. All Rights Reserved.

Objectives

- Understand the different deployment models of the agents
- Understand the pros and cons of each deployment model

© 2011-24 National University of Singapore. All Rights Reserved.

Topics

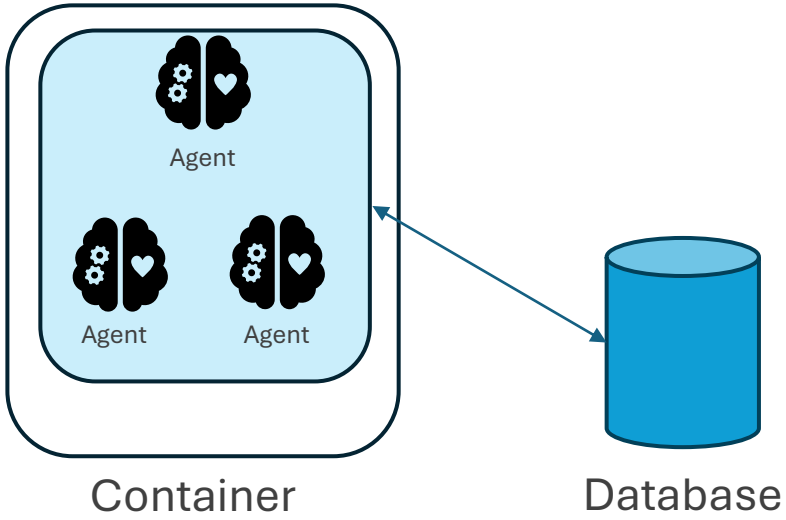
- Architectural styles and components
- **Deployment models**
- Physical architectures for AI Agentic Solution

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Single Container

- All agents deploy in one container image
- No distributed setup—everything is in one unit
- For Docker implementation, deployed as one container image
- Deployed using tools like docker run, docker-compose (with 1 service), or Kubernetes (with 1 pod/1 container)



© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Single Container – When to use it ?

- Small/simple apps
- Development or testing environments
- Prototypes or MVPs where speed matters more than scalability
- Fixed number of users where scalability doesn't matter

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Single Container – Pros

- Simple architecture don't need to design for fault tolerance like those in the distributed one
- Easy to setup
- Low Latency – as all agents are under 'one roof'

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Single Container – Cons

- No granular scalability – can't scale one particular agent as all are bundled into one VM

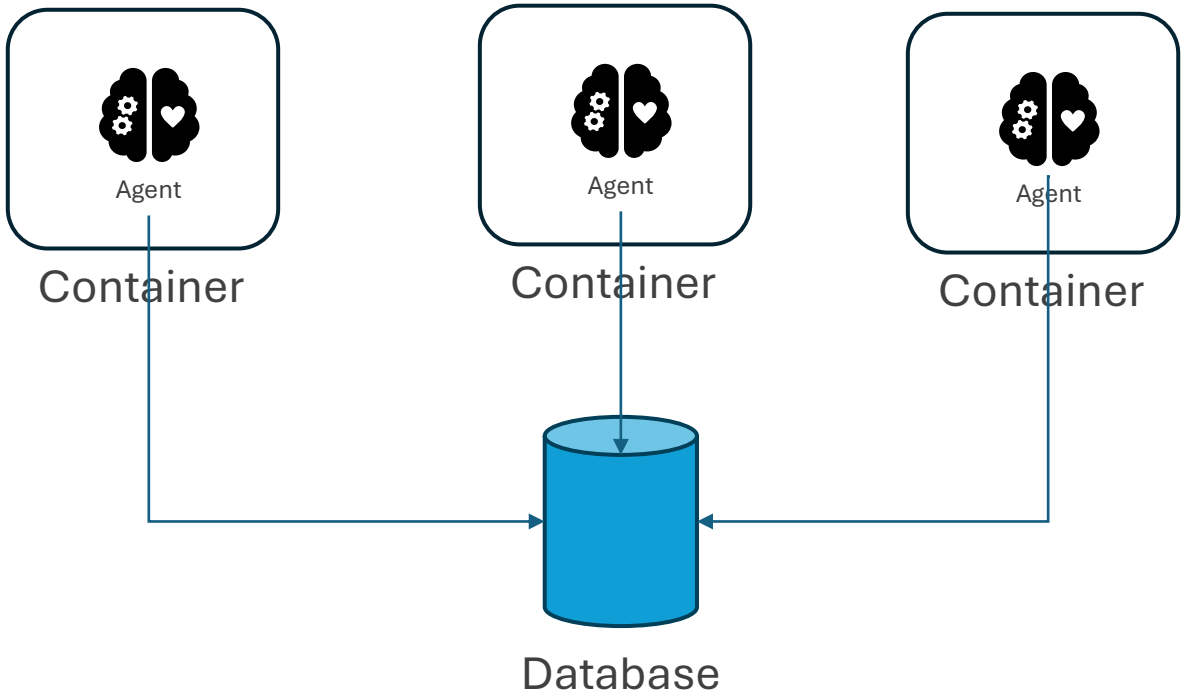
- Cascading failure – as all agents or components are bundled together should one or more component fail then it might cause all other non relevant ones to stop working

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Multiple Containers

- One agent per container
- For Docker implementation, deployed as one container image
- Deployed using tools like docker run, docker-compose (with 1 service), or Kubernetes (with 1 pod/1 container)



© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Multiple Containers – When to use it ?

- Fine independent granular scaling is needed
- Multiple teams collaboration - For example one agent per development team
- Polyglot environment - Some agents or components need to be developed in different programming languages
- High reliability – one agent or component that fails will not affect other as they are deployed in different container

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Multiple Containers – Pros

- Modularity & separation of concerns
  Each service/agent (API, worker, vector DB, broker, monitor) is isolated for clearer ownership and faster iteration
- Independent scaling
  Scale hot spots only (e.g., more inference workers, fewer orchestrators) → better utilization and cost control
- Fault isolation & resiliency
  One container can crash/restart without taking down the rest; easier rolling updates and canary releases
- Polyglot flexibility
  Mix languages/runtimes and swap components (e.g., change vector store) with minimal ripple
- Security boundaries
  Per-container least privilege, network policies, and read-only filesystems reduce blast radius

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Multiple Containers – Cons

- **Operational complexity**
  Orchestration (K8s), service discovery, secrets, and network policies add steep learning and maintenance overhead
- **Distributed debugging**
  Tracing cross-service failures, timeouts, and data mismatches is harder than in a monolith.
- **Network & serialization overhead**
  Extra hops (REST/gRPC/queues) can add latency and cost versus in-process calls.
- **Complex CI/CD**

  More artifacts to manage
  Multiple images, versions, and dependency SBOMs increase CI/CD complexity
- **Cost**

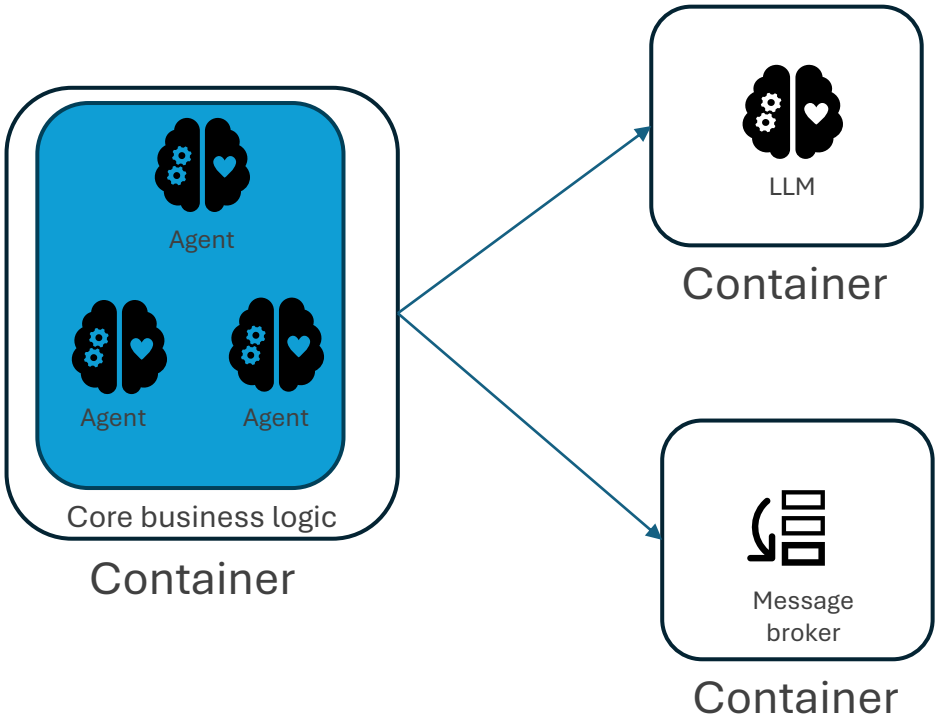
  Higher setup cost. You will need additional components like load balancer etc

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Hybrid Model

- Single and Multiple containers combined
- Core business logic in one container and other in multiple containers



© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Hybrid Model – When to use it ?

- **Edge computing (EC) in the IoT environment**

  EC will host the core business logic and integrate with other LLM tools, database etc in cloud environment

- **Legacy reason**

  Legacy and matured system is in monolith architecture but wish to extend with modern and AI functionality

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Hybrid Model – Pros

- **Latency control**
  Better than pure multi-containers model at least the core business logic or the edge is in one model
- **Lesser risk**
  Less risky as the legacy is not replaced.
- **Incremental modernization**
  Keep legacy monoliths running while adding cloud microservices/agents (“strangler”).
- **Performance locality**
  Compute near data sources (factories, hospitals, branches) to cut backhaul.

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Hybrid Model – Cons

- **Operational complexity**
  More networks, IAM domains, secrets, certs, and infra-as-code to manage
- **Data consistency & sync**
  Bi-directional replication, schema/version drift, conflict resolution could be hard
- **Higher fixed costs on-prem**
  CapEx, capacity planning, power/cooling, and hardware lifecycle

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Microservice - Characteristics

- **Atomic**

  System is broken down into small, independent and loosely coupled service

- **Single Responsibility**

  Each service focuses on a specific business function (e.g., authentication, payment, product)

- **Decentralized Data Management**

  Each microservice usually has its own database or data storage, reducing dependencies. However, it is not feasible to have one physical database for each service most of the time it is logically separated

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Microservice - Characteristics

- **Communication**

  Microservices communicate with each other via lightweight protocols, often HTTP/REST, gRPC, or messaging systems like Kafka or RabbitMQ

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Microservice - AI Agentic

- **AI Agent as a microservice**

  AI Agents that are independent, focus on a specific function can be architected as microservices.

  One agent -> One microservice

  Many agents -> One microservice

  For instance, AI Agents are parts of a workflow process might merge into one microservice.

- **Domain Driven Design (DDD)**

  Adopt DDD approach for better classification of microservices

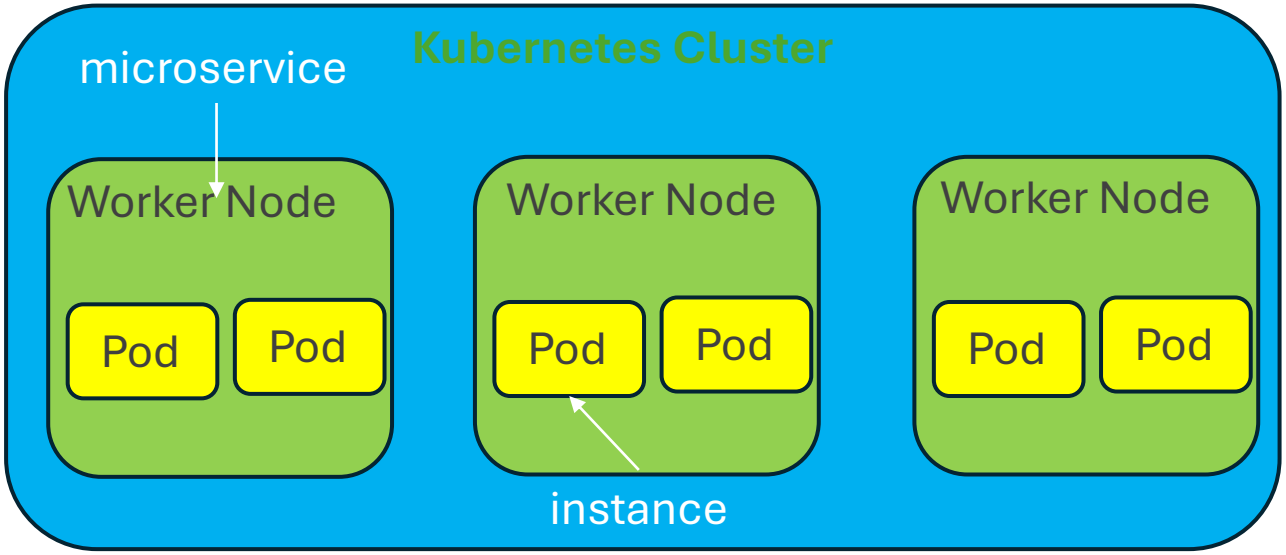
© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Microservice Tool – Kubernetes

- **Docker Orchestration**

  Open source container orchestration platform developed by Google. Now maintained by Cloud Native Computing Foundation (CNCF)



© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Kubernetes - Benefits

- **Automated Deployment & Scaling**

  Launches containers and scales them up or down based on demand
- **Self-Healing**

  Restarts failed containers, replaces them, and reschedules workloads automatically
- **Load Balancing & Service Discovery**

  Distributes traffic evenly and makes services accessible without manual configuration
- **Configuration & Secret Management**

  Stores and manages configuration data securely
- **Portability**

  Works across different environments: on-premises, cloud (cloud agnostic), or hybrid

© 2011-24 National University of Singapore. All Rights Reserved.

Deployment Models

Kubernetes - Cons

Complexity

- Steep learning curve, especially for beginners.
- Requires understanding of pods, services, deployments, namespaces, etc.

Operational Overhead

- Needs ongoing management (upgrades, monitoring, logging, security hardening).
- Running a cluster at scale often requires a dedicated DevOps/SRE team.

Resource Intensive

- Kubernetes itself consumes CPU, memory, and storage.
- Can be costly to operate, especially for small teams or low-budget projects.

Not Always Needed

- Overkill for simple apps or monoliths.
- Alternatives like Docker Compose or managed container services (AWS ECS, GCP Cloud Run) may be easier.

© 2011-24 National University of Singapore. All Rights Reserved.

Summary.

- The different deployment models
  - Single container
  - Multiple container
  - Hybrid
- Microservice Architecture
  - Characteristic
  - Pros and Cons
- Kubernetes to implement Microservice architecture.

© 2011-24 National University of Singapore. All Rights Reserved.

Topics

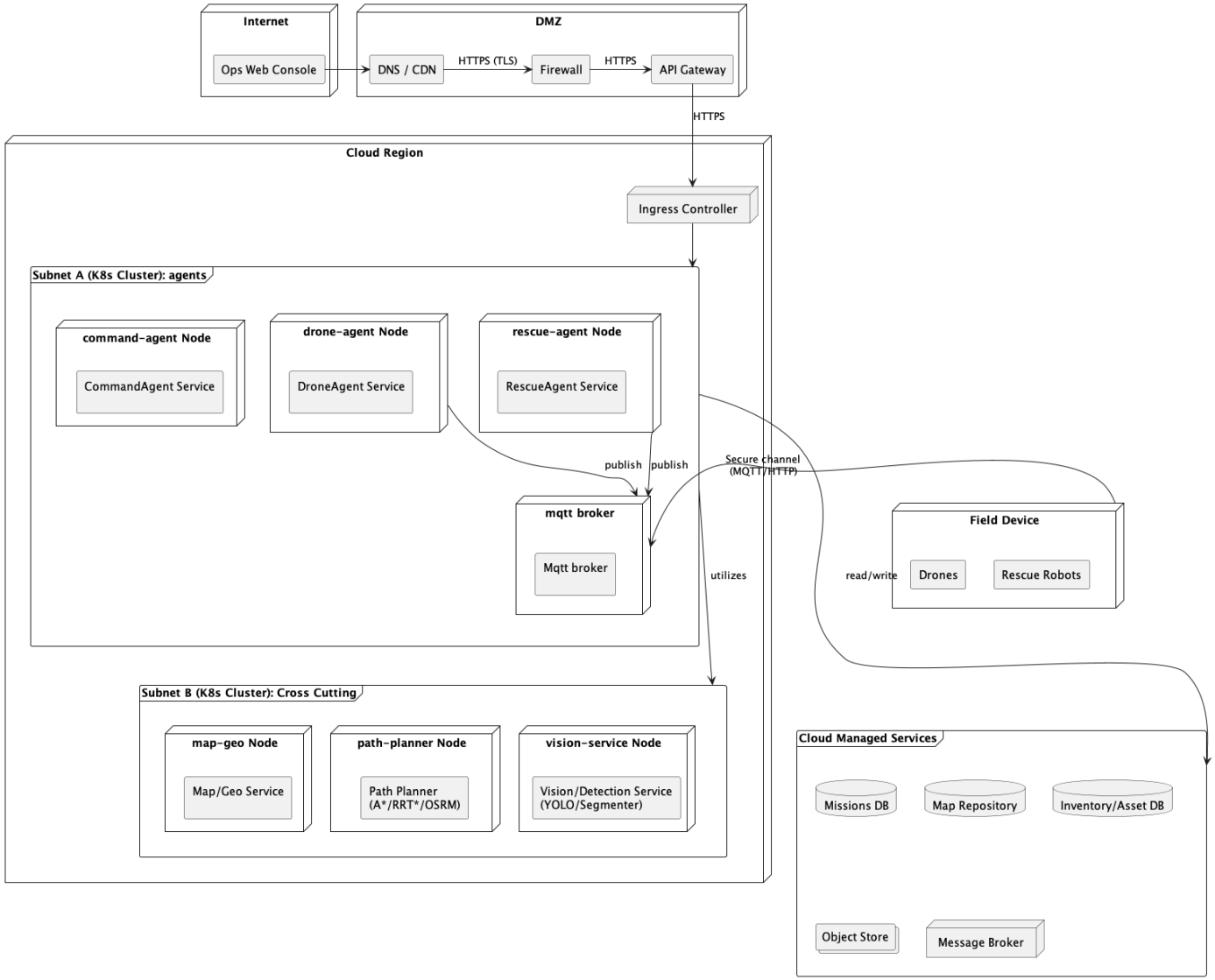
- Architectural styles and components
- Deployment models
- **Physical architectures for AI Agentic Solution**

© 2011-24 National University of Singapore. All Rights Reserved.

Now is time to piece everything together

© 2011-24 National University of Singapore. All Rights Reserved.

Physical Architecture - Overview



© 2011-24 National University of Singapore. All Rights Reserved.

Physical Architecture - Overview

- **Subnet**

  One subnet per K8s cluster

- **Agent**

  Each agent is a microservice and deployed as one worker node. Each node could have auto-scaled pods

- **MQTT Broker**

  The MQTT broker is the communication channel among the agents and the UAV(drones and rescue robots). Assuming the UAV could support MQTT protocol.

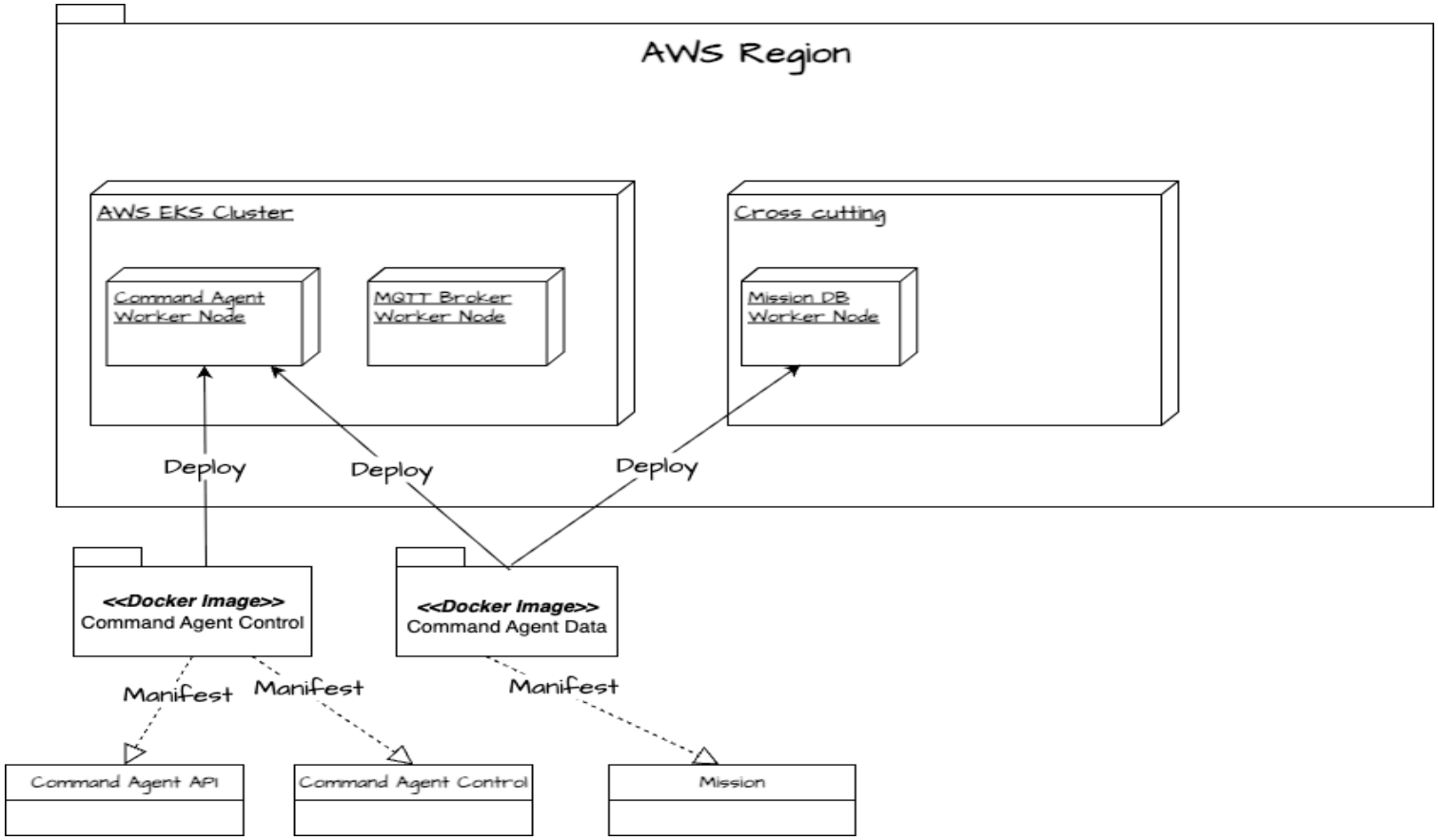
- **Cross cutting Cluster**

  These are the ML models that performs path planning and vision detection. A map service. These are used across by all agents

© 2011-24 National University of Singapore. All Rights Reserved.

Physical Architecture

Deployment Component Diagram – Command Agent



© 2011-24 National University of Singapore. All Rights Reserved.

Reference Architecture

Use Case – Intelligent Customer Support

- An intelligent customer support solution for a large retailer.

- Showcases how multiple specialized agents can collaborate to provide comprehensive, personalized support to customers

© 2011-24 National University of Singapore. All Rights Reserved.

Reference Architecture

Use Case – Intelligent Customer Support

Agent Involved:

- Supervisor Agent
- Order Management Agent
- Product Recommendation Agent
- Troubleshooting Agent
- Personalization Agent

© 2011-24 National University of Singapore. All Rights Reserved.

Reference Architecture

Use Case – Intelligent Customer Support

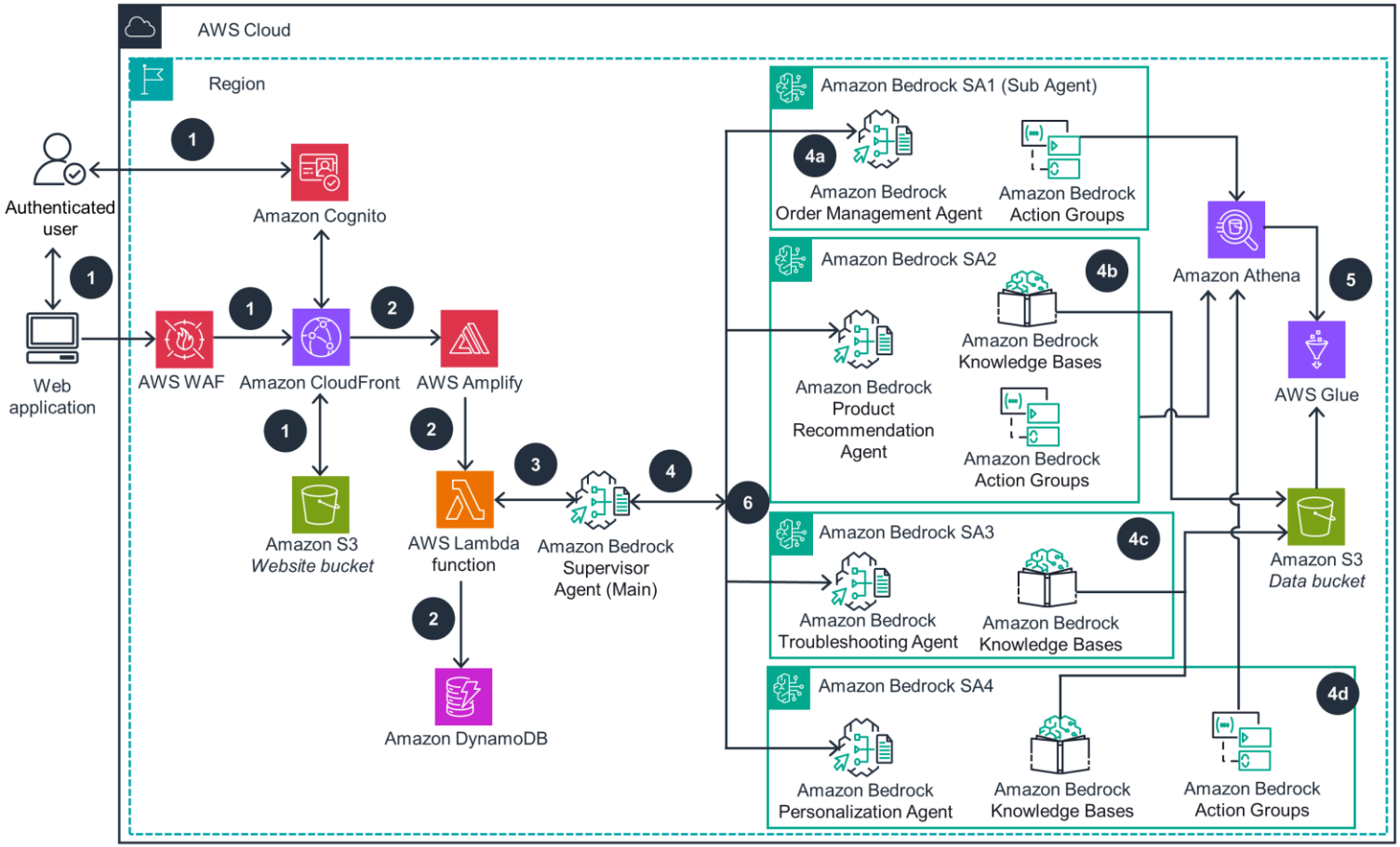
Technology Stack

Layer	Technology
Front End	ReactJs (Web based), Cloud Front, AWS S3 Bucket (host the pages, JS)
Back End	AWS Lamda function
Agent layer	AWS Bedrock
Communication Styles	Web Socket API between front end to backend
Data storage	AWS Athena, AWS S3 Bucket

© 2011-24 National University of Singapore. All Rights Reserved.

Reference Architecture - AWS

Reference Architecture - AWS



Source: AWS Solution Library

© 2011-24 National University of Singapore. All Rights Reserved.

Physical Architecture

1. Users access the web application via WAF and CloudFront. Authentication process kick in and performed by Cognito.

2. After authentication, users' requests will be routed to AWS Amplify route to a Lambda function. Lambda function will store the user session in Dynamo DB.

3. The AWS Lambda function processes the incoming requests and communicates with the Amazon Bedrock Supervisor Agent (Main).

4. Supervisor Agent will analyse the user query's intent and determine which agent to 'arrow'. This central orchestrator maintains context across the conversation and ensures requests are handled by the most suitable sub agent.

© 2011-24 National University of Singapore. All Rights Reserved.

Physical Architecture

4. Supervisor Agent will analyses the user query's intent and determine which agent to 'arrow'. This central orchestrator maintains context across the conversation and ensures requests are handled by the most suitable sub agent.
   a. Order Management Agent
   b. Product Management Agent
   c. Troubleshooting Agent
   d. Personalisation Agent
5. Sub agents construct and execute SQL queries against Amazon
6. After having information from databases and knowledge bases, the sub agents generates a comprehensive response, which is then sent back through the Supervisor Agent to the Lambda function, AWS Amplify, and finally to the user's web interface.

© 2011-24 National University of Singapore. All Rights Reserved.

Summary.

- Define the physical architecture overview of the Agentic AI
- Detail component deployment
- Reference architecture
  - AWS

© 2011-24 National University of Singapore. All Rights Reserved.

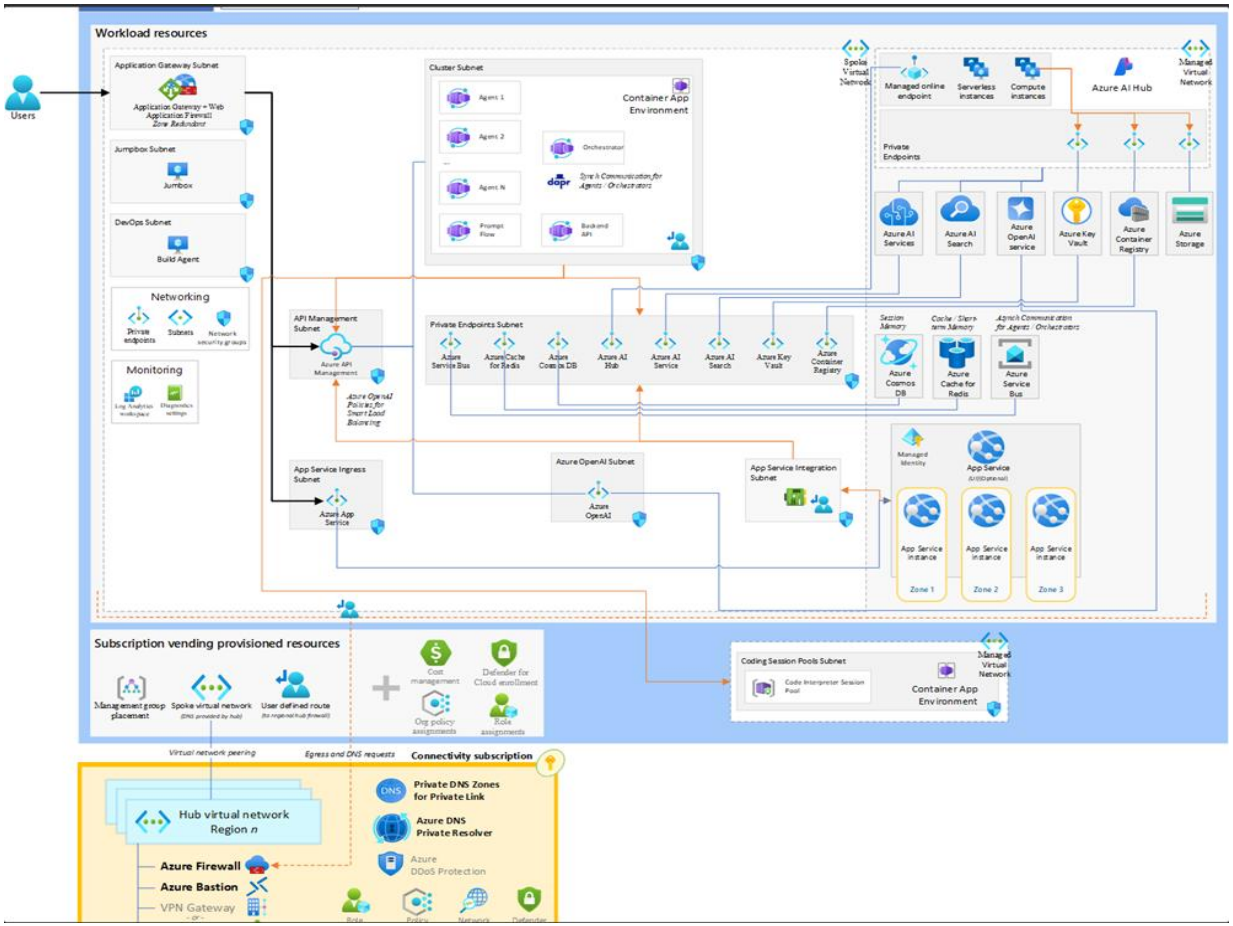
ARCHITECTING AGENTIC AI SOLUTIONS

APPENDIX

© 2011-25 National University of Singapore. All Rights Reserved.

Physical Architecture

Reference Architecture - Azure



Source: https://medium.com/@harshraj19/an-introduction-to-agentic-azure-ai-architecture-a-beginners-guide-9622b34b04da

© 2011-24 National University of Singapore. All Rights Reserved.